PHASE 4 / MODEL DEVELOPMENT / 01

Machine Learning Model Development

Smart Parking System
Training, Evaluation & Local Prediction

Persistence | Linear Regression | Random Forest | 15-minute horizon

Prepared 11 November 2026. The required execution target is this Windows laptop, including future Streamlit inference. No cloud or SSH runtime is used. All observations remain explicitly synthetic.

ACTUAL MODEL VERIFICATION AVAILABLE

Actual model fitting, validation comparison, final test evaluation and trained-artifact reload verification succeeded locally. Persistence was selected by the predeclared validation MAE criterion. Learned models remain saved for comparison; a synthetic result is not a real-world accuracy claim.

Verification item	Current recorded result
Frozen Phase 3 audit	Dataset hashes, target alignment, feature list and stored source/report hashes match.
Final native/software tests	21 actual tests; passed=True. Saved full-data models also passed independent reload verification.
Local native preflight	Passed=True in the approved isolated local Windows environment.
Actual project model evaluation	Available in saved results
Actual trained-model reload	PASS
Git delivery policy	One meaningful file per commit; one final push after all verification and PDF review.

This report reads actual model results, locked validation selection, final native tests and saved-model verification evidence. Earlier blocked preflights and the verification-only recovery are retained for traceability. No model or hyperparameter was changed after the test partition was opened.

PHASE 4 / MODEL DEVELOPMENT / 02

Objectives, frozen data and features

Predict total occupied slots exactly 15 minutes ahead, bounded by the physical capacity of twelve spaces. Compare a strong current-occupancy persistence baseline with ordinary Linear Regression and a nonlinear Random Forest. Model selection must use validation outcomes only; the held-out test period supplies final reporting after the choice is locked.

Partition	Rows	Forecast origins, IST	Last target, IST
Training	49360	21 Sep 00:00 - 25 Oct 06:39	25 Oct 06:54
Validation	10577	25 Oct 06:55 - 1 Nov 15:11	1 Nov 15:26
Testing	10578	1 Nov 15:27 - 8 Nov 23:44	8 Nov 23:59

The frozen processed table has 70,515 rows, 42 columns, no missing values and 37 predictors. Raw history has 80,640 minute observations. Target timestamps and values were audited against the exact raw t+15 row. Existing fifteen-minute boundary purges keep preceding labels strictly before the next partition origin. No data is regenerated or repartitioned.

Predictor group	Frozen columns / information
Current state and counts	P1-P12; total_occupied; arrivals; departures.
Calendar	hour, minute, day_of_week, is_weekend, time_sin/cos, week_sin/cos.
Occupancy lags	1, 5, 15, 30, 60, 1,440 and 10,080 minutes.
Past rolling windows	Means over 5/15/60/1,440 minutes; population standard deviations over 15/60; windows end at t-1.
Change	Current total minus the fifteen-minute lag.

Only FEATURE_COLUMNS enters X. Raw timestamp, data_source, target_timestamp, occupied_15min_ahead and split are excluded. Current t observations are assumed available at forecast time. Future demand and scheduled departure parameters are not inputs. No live Wokwi/SQLite data is merged or altered.

PHASE 4 / MODEL DEVELOPMENT / 03

Models, preprocessing and configurations

Persistence baseline

```
prediction(t+15 | t) = total_occupied(t)
```

No fitting is required. It is evaluated on exactly the same validation/test rows as learned models, and remains eligible to be the selected model. Synthetic minute occupancy is highly persistent, so beating this baseline must be demonstrated rather than assumed.

Linear Regression

```
z_j = (x_j - training_mean_j) / training_scale_j
y_hat = intercept + sum_j(beta_j * z_j)
Fit: minimize sum_training((y - y_hat)^2)
```

A Pipeline contains StandardScaler followed by LinearRegression, fitted on float64 training rows only. Scaler sample count and means are checked against training data. Validation never updates preprocessing. Redundant frozen columns (slot sum and lag/change identities) can make the design rank deficient; solver rank is recorded. Coefficients should not be interpreted as causal effects. No frozen features are removed to improve scores.

Random Forest Regressor

```
forest_prediction(x) = mean(tree_1(x), ..., tree_100(x))
```

Parameter	Predeclared setting
n_estimators / random_state	100 / 42
bootstrap / max_samples	True / 0.8
max_features / n_jobs	0.8 / 2
max_depth	8 or 12
min_samples_leaf	10 or 30

Four depth/leaf combinations are fitted on training only and scored on the same validation rows. Select the best forest by clipped MAE, then RMSE, then fixed grid order. Only the best forest is retained. Native numerical thread pools are limited to avoid nested oversubscription. All four configurations were actually fitted and validated. The chosen forest has depth 8 and minimum leaf size 30; the overall MAE-selected model is persistence. Observed linear rank is 34 of 37 predictors.

PHASE 4 / MODEL DEVELOPMENT / 04

Leakage protection, comparison and metrics

Stage	Implemented information boundary
Dataset check	Verify frozen raw/processed hashes and exact feature/horizon contract.
Partition read	Read only training or validation row bounds; do not parse test labels during tuning.
Fit	Training-only scaler and learned model coefficients/trees.
Validation	All models use identical X/y rows; forest grid is fixed beforehand.
Selection lock	Write validation_selection.json, selected model, scores/configurations and row hashes before opening test.
Final evaluation	First test read occurs after lock; score all three models on identical rows. No refit or reselection.
Persistence verification	Save one bundle; reload every model; check the selection-lock hash and bounded predictions.

```
MAE  = mean(abs(y - prediction))
RMSE = sqrt(mean((y - prediction)^2))
R2   = 1 - SSE / SST
SSE  = sum((y - prediction)^2)
SST  = sum((y - mean(y))^2)
delivered_prediction = clip(raw_prediction, 0, 12)
```

Primary selection is clipped validation MAE, then clipped validation RMSE, then a fixed simplicity order: persistence, linear, forest. Test performance cannot change this selection. All models keep their training-only fits; there is no train+validation refit that would make the comparison unequal. Scikit-learn recommends train-only preprocessing and pipelines to reduce leakage. [1]

Both raw and clipped metrics and clipping frequency are reported when actual execution exists. Clipping is a fixed predeclared physical constraint, applied equally before selection. No rounding is applied: regression outputs are expected counts. R2 is explicitly undefined/null for constant targets. Negative R2 values are valid and will not be hidden.

PHASE 4 / MODEL DEVELOPMENT / 05

Actual tests, validation and test results

Executed verification	Actual evidence
Contract suite	21 passed; 0 failures; 0 errors; 0 skips; 1.453 seconds.
Input/output contract	Named-column reordering; missing/extra/invalid values; slot-sum consistency; output clipping.
Metric/selection contract	Hand-calculated MAE/RMSE/R2; undefined constant R2; alignment/range checks; validation-only selector and simplicity tie-break.
Partition and provenance	Requested validation row bounds, frozen checksums and predictor allowlist.
Saved full-data model verification	All 28 independent checks passed, including each model on identical validation/test row hashes and bounded reload predictions.
Native tests	Actual train-only scaler, identical validation rows, seeded forest repeatability, sklearn metric agreement and trained bundle reload passed.

Validation comparison

Model	MAE	RMSE	R2
persistence	0.339983	0.684103	0.950447
linear_regression	0.414362	0.656363	0.954385
random_forest	0.436999	0.680876	0.950914

Test comparison

Model	MAE	RMSE	R2
persistence	0.307714	0.648092	0.945254
linear_regression	0.373729	0.613500	0.950942
random_forest	0.394675	0.641002	0.946445

Selected model: persistence. The primary criterion is validation MAE. Linear Regression has lower test RMSE, but this did not change selection. All metrics describe synthetic data only. Reload/recovery details are documented separately; no test-driven retuning occurred.

PHASE 4 / MODEL DEVELOPMENT / 06

Verified local Windows execution

The original project environment failed native imports with an Application Control denial (_partition_nodes, earlier _openmp_helpers/_sag_fast). The user then approved an isolated official-PyPI installation trial. The completed isolated runtime passed imports, dependency checks, fitting, evaluation and saved-model reloads without a policy change.

Local diagnostic	Observed result
Python / wheel compatibility	Python 3.12.10 x64; wheel tag cp312-cp312-win_amd64 matches.
Dependency check	pip check: No broken requirements found.
Native file integrity	Three blocked module files match installed wheel RECORD hashes. _partition_nodes Authenticode status: NotSigned.
Code Integrity event	3077 confirms PyArrow lib.cp312-win_amd64.pyd denied by Enterprise signing policy {0283ac0f-fff1-49ae-ada1-8a933130cad6}.
Evidence limit	No separate sklearn denial event correlated. The PyArrow policy event must not be described as a direct sklearn policy attribution.
Current process signature flags	Read-only query returned zero; no flags were changed.
Verified runtime	%LOCALAPPDATA%/SmartParking/ml-runtime, Python 3.12.10 x64, locally executable Windows packages.

Microsoft describes event 3077 as an enforced App Control block. The original failure remains historical evidence. The approved clean runtime succeeded; the exact cause of the original environment failure has not been conclusively established. No claim is made that the original environment was repaired. [2]

Approved installation actually performed

Existing Python created the separately approved environment. Only exact pinned binary packages from https://pypi.org/simple were installed with isolated pip configuration. A first probe found missing cloudpickle, not a policy denial; cloudpickle, python-dateutil and tzdata were added at existing project pins. The completed native probe and pip check passed. No policy, trust exception or DLL relocation was used.

The original project venv was never an installation target; its freeze still matches frozen requirements. Run training and future inference with the isolated interpreter. This resolves the local execution requirement without cloud/SSH or changing frozen Phase 1/2/3 artifacts.

PHASE 4 / MODEL DEVELOPMENT / 07

Prediction interface and local reproducibility

The locally verified ParkingPredictor interface accepts a DataFrame containing exactly the 37 predictor columns, one feature dict, or a list of feature dicts. It validates names, finite numeric values, current binary slots and their total. It reorders named columns safely and clips every output to 0-12. Unnamed matrices are rejected at the public interface.

```
import json
from pathlib import Path
from ml.phase4_models import load_predictor
results = json.loads(Path("ml/artifacts/phase4/model_results.json").read_text())
predictor = load_predictor(
    "ml/artifacts/phase4/parking_models.joblib",
    expected_sha256=results["artifact"]["sha256"],
)
predicted_counts = predictor.predict(feature_rows)
# Optional explicit comparison:
# predictor.predict(feature_rows, "persistence")
```

The verified 851,974-byte compressed bundle contains the three model definitions, fitted scaler, chosen forest, selection identifier, feature order, 15-minute horizon, seed and data/selection hashes. Joblib requires a compatible permitted local runtime; load only trusted artifacts because a checksum is not an authenticity or pickle-safety guarantee. [3]

Verified local runtime commands

```
$phase4Python = Join-Path $env:LOCALAPPDATA `
  'SmartParking\ml-runtime\Scripts\python.exe'
& $phase4Python -B -m ml.tools.phase4_test_runner --native `
  --output reports/evidence/phase4_native_followup.json
& $phase4Python -B -m ml.tools.verify_phase4 --run-dir ml/artifacts/phase4 `
  --output reports/evidence/phase4_verification_followup.json
```

Use a new output/evidence path for a deliberate rerun; files are never silently replaced. Do not retune using test outcomes. Future Streamlit integration must use the same permitted local runtime and sufficient causal history. Missing or UNKNOWN real-slot inputs must not be replaced with synthetic values to manufacture a forecast.

The original ml/train.py placeholder is preserved because it is included in frozen Phase 3 hashes. All new training code is in train_phase4.py; Phase 3 feature/data code and notebooks remain unchanged.

PHASE 4 / MODEL DEVELOPMENT / 08

Acceptance, storage, assumptions and evidence

Critical requirement	Current status
Native tests and prediction contracts	PASS: final 21 tests, including actual fitting/reload.
Frozen Phase 1/2/3 preservation	Phase 3 hashes intact; no frozen datasets/code/reports modified.
Approved local Windows runtime	PASS: isolated imports, fitting/evaluation and reload succeeded.
Actual fitting, validation and test comparison	Actual results available
Actual trained model and reload	PASS
Phase 4 complete	YES
Per-file commits and final push	Performed only after final checks and visual PDF review; confirmed at delivery.

Current data/model directory storage: 8,190,119 bytes (8.190 MB). The limit remains 50,000,000 bytes. The runner checks the total after saving outputs. No duplicate datasets, prediction-row export or notebook generator is used. The directory total includes the one fitted bundle, frozen datasets and supporting model metadata; runtime/dependency installation storage is separate.

Assumptions/limits: synthetic and uncalibrated demand; highly persistent minute occupancy; redundant predictors; single chronological holdout; only four forest configurations; no real-site, sensor-noise, live-MQTT or dashboard validation. A synthetic winner may not generalize. No confidence intervals or deployment accuracy claims are supplied.

Evidence and continuation files

Historical preflights/readiness JSON preserve initial failures. phase4_isolated_complete_preflight.json and phase4_installation_trial.json record the approved runtime. phase4_native_tests_final.json records 21 passes; phase4_model_verification.json records 28 passes. model_results.json and validation_selection.json contain actual metrics/configurations. The final checklist and phase4_final_checks.json record preservation and PDF acceptance.

Primary references

[1] Scikit-learn leakage and preprocessing
https://scikit-learn.org/stable/common_pitfalls.html

[2] Microsoft App Control event explanations
https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/operations/event-id-explanations

[3] Scikit-learn model persistence
https://scikit-learn.org/stable/model_persistence.html

Regenerate this PDF using reports/generate_phase4_report.py in the bundled ReportLab environment. It does not install project dependencies. This document reports actual synthetic fitting/evaluation and its limitations; it is not a certificate of real-world performance.

PHASE 4 / MODEL DEVELOPMENT / 09

Validation search, baseline and recovery

Forest depth	Minimum leaf	Validation MAE	Validation RMSE
8	10	0.445964	0.695580
8	30	0.436999	0.680876
12	10	0.484498	0.729381
12	30	0.458959	0.700878

The best forest is depth 8 / minimum leaf 30. It still has higher validation MAE than persistence (0.436999 versus 0.339983), so the saved selection is persistence. Linear validation MAE is 0.414362. Test outcomes never changed this decision. All three models are retained in one bundle for transparent comparison.

Partition / model	Raw MAE	Clipped MAE	Clipped rows
validation / persistence	0.339983	0.339983	0 (0.00%)
validation / linear_regression	0.423693	0.414362	1302 (12.31%)
validation / random_forest	0.436999	0.436999	0 (0.00%)
test / persistence	0.307714	0.307714	0 (0.00%)
test / linear_regression	0.384557	0.373729	1590 (15.03%)
test / random_forest	0.394675	0.394675	0 (0.00%)

Verification-only recovery, with no retuning

The first full-data run locked persistence before test access and saved its bundle. An exact-equality assertion then rejected parallel forest summation differences. Two reloads differed by at most 1.7763568394002505e-15 slots; persistence and linear predictions were exactly equal. Reload agreement now uses absolute tolerance 1e-12, relative tolerance zero, with a test rejecting materially larger differences.

The original bundle and selection file were not overwritten. Finalization read that same artifact and checked its configuration, training-only scaler, locked validation scores and hashes. Test metrics were recomputed for recovery and independently reverified. No fitting, hyperparameter change, reselection or test-driven feature change occurred. The saved results explicitly disclose this recovery; no single-computation claim is made.

PHASE 4 / MODEL DEVELOPMENT / 10

Reproducibility, storage and Phase 5 handoff

Artifact	Actual bytes / purpose
parking_models.joblib	851,974; all three models and required preprocessing
validation_selection.json	5,519; pre-test selection lock and four candidate results
model_results.json	4,854; actual raw/clipped metrics, row identities and reload evidence
Frozen data + model directory total	8,190,119 bytes; below 50,000,000

Model SHA-256:
cfc2adc80cc949e3c097530bca500079b857eee8b931be1e6d7de522f50335e1

Selection-lock SHA-256:
1c87ab156a4189b20e825a73f77bd3373d5a63ec25178705ea838360376eeecb

Runtime versions: scikit-learn 1.9.1, numpy 2.5.4, pandas 3.1.1, scipy 1.18.1, joblib 1.6.0, threadpoolctl 3.8.0. Full approved package pins are in ml/phase4_requirements.txt. There is no cloud, SSH, new Python installation, security exception or modification of the old project environment.

Deliberate new training run, not an overwrite

```
$phase4Python = Join-Path $env:LOCALAPPDATA `
  'SmartParking\ml-runtime\Scripts\python.exe'
& $phase4Python -B -m ml.train_phase4 `
  --output-dir ml/artifacts/phase4_reproduction
& $phase4Python -B -m ml.tools.verify_phase4 `
  --run-dir ml/artifacts/phase4_reproduction `
  --output reports/evidence/phase4_reproduction_checks.json
```

These commands intentionally create new artifacts and are not run merely to inflate activity or select another model using the test period. Reproducibility tests used small seeded fixtures; exact bitwise forest prediction is not promised under parallel floating summation. Reload correctness uses the strict documented numerical tolerance.

Future Streamlit integration

Load the trusted bundle with its recorded checksum and call predictor.predict on named, complete causal features. All three models share the same input contract and output range. Use a compatible approved local runtime for the future dashboard; Streamlit installation and integration are a separate authorized phase. If live slots or lookback history are unavailable, expose forecast unavailability rather than substituting synthetic records. No Phase 5 code is added here.